TP 1 : Lecture et Traitement des Données Vidéo

1.1 Objectif pédagogique du TP

Ce travail pratique a pour objectif d'explorer la manière dont une architecture matérielle spécialisée, en particulier le processeur de signal numérique DSP TMS320C6416, permet d'implémenter efficacement un algorithme de compression vidéo conforme à la norme H.264/AVC. L'étude vise à analyser l'interaction entre le matériel et le logiciel dans un contexte de traitement en temps réel, en s'intéressant notamment à la gestion du cache, aux transferts de données via le DMA et à l'utilisation de la mémoire interne LL2RAM, dont la combinaison contribue à optimiser les performances globales du système. Ce TP s'inscrit dans une démarche d'application concrète des principes théoriques du codage vidéo, en mettant en relation la compression, l'optimisation et le calcul numérique avec le fonctionnement réel d'un DSP.

1.2 Le DSP : rôle et applications

Un processeur de signal numérique (DSP) est un microprocesseur optimisé pour le traitement rapide des signaux numériques. Sa conception repose sur l'exécution efficace des opérations arithmétiques et logiques nécessaires aux calculs intensifs en temps réel. Grâce à une architecture favorisant le parallélisme et la manipulation directe des flux de données, le DSP est utilisé dans des domaines variés tels que les télécommunications (modems DSL, RTC), les systèmes embarqués (téléphones portables, GPS), la multimédia (lecteurs MP3, traitement audio/vidéo) ou encore la reconnaissance de signaux complexes. Le fonctionnement général d'un système à DSP est illustré dans la Figure 1.1, qui présente les différentes étapes du traitement : la conversion du signal analogique vers le numérique via le filtre anti-repliement et l'ADC, le traitement numérique effectué par le DSP, puis la reconversion vers un signal analogique grâce au DAC et au filtre de reconstruction.

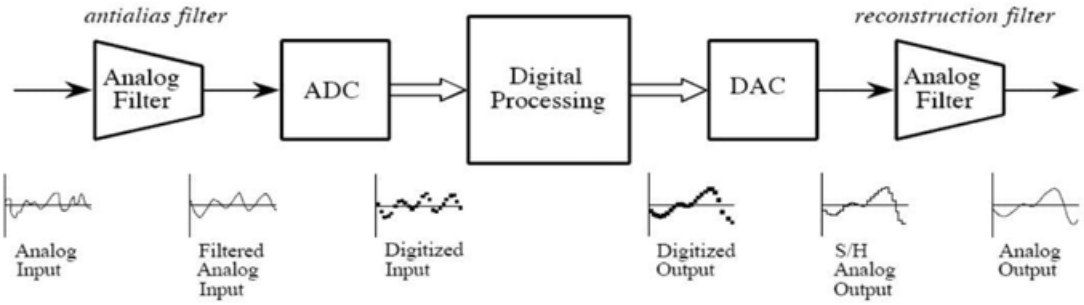


FIGURE 1.1 – Schéma fonctionnel d'un système à processeur de signal numérique (DSP) [2]

Ses principales forces résident dans la rapidité du filtrage numérique, la précision des calculs et la fiabilité de ses performances, bien que sa programmation et son optimisation soient souvent plus complexes que celles d'un microprocesseur généraliste. Une vue d'ensemble de ces différents avantages et inconvénients est présentée dans le Tableau 1.1, qui résume les points forts du DSP ainsi que les principales limites associées à son utilisation.

TABLEAU 1.1 – Synthèse des avantages et inconvénients du DSP

Avantages	Inconvénients
Traitement rapide des signaux numériques.	Programmation complexe et optimisation difficile.
Bonne efficacité énergétique.	Mémoire interne limitée.
Idéal pour les applications temps réel.	Moins flexible pour des tâches générales.
Haute précision et fiabilité.	Coût de développement initial élevé.

1.3 Le standard de compression H.264/AVC

Le standard H.264/AVC [1] marque une avancée majeure dans le codage vidéo numérique, en offrant une forte réduction du débit binaire tout en maintenant une excellente qualité d'image. Cette performance repose sur l'exploitation optimale des redondances spatiales et temporelles entre les trames, à l'aide de techniques de prédiction, de transformée, de quantification et de codage entropique comme illustré par la Figure 1.2. Cependant, cette efficacité implique une complexité algorithmique élevée, notamment dans les modules de prédiction et de compensation, qui exigent une puissance de calcul importante. Pour garantir un traitement en temps réel (25 à 30 images par seconde), l'utilisation d'une architecture matérielle spécialisée, telle qu'un processeur de signal numérique (DSP), s'avère indispensable.

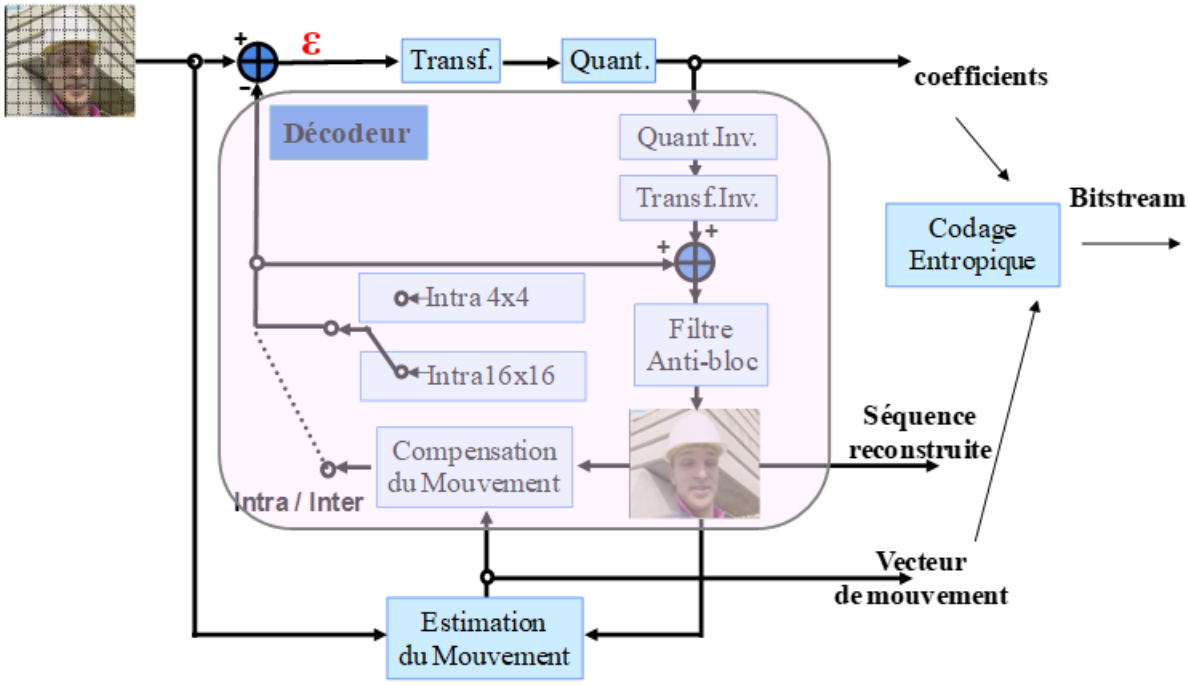


FIGURE 1.2 – Schéma fonctionnel d'un encodeur vidéo hybride typique H264/AVC

1.3.1 Principes de la compression H.264 (Prédiction intra-image)

Le standard H.264 repose sur une approche de réduction des redondances spatiales à l'intérieur d'une même image, à travers la prédiction intra-image. Cette technique consiste à estimer les valeurs des pixels à partir de leurs voisins déjà codés, afin de ne transmettre que la différence (signal résiduel) entre la valeur réelle et la valeur prédite comme illustré dans la Figure 1.3. Dans le cas des zones d'image régulières, la prédiction INTRA-16x16 est privilégiée. Elle s'applique aux composantes de luminance (Y) d'un macrobloc 16×16 pixels et peut être réalisée selon plusieurs modes de calcul :

— Verticale, où les pixels sont prédits à partir des valeurs situées au-dessus du bloc,

— Horizontale, basée sur les pixels adjacents à gauche,

— DC, qui utilise la moyenne des valeurs voisines,

— Plan, mode le plus complexe, qui ajuste une surface plane en fonction des variations locales.

Cette approche permet de réduire considérablement la quantité de données à coder tout en maintenant une qualité d'image élevée. Le signal résiduel, c'est-à-dire la différence entre l'image originale et la prédiction, est ensuite soumis à une transformée et une quantification, optimisant ainsi le taux de compression sans dégradation visuelle perceptible.

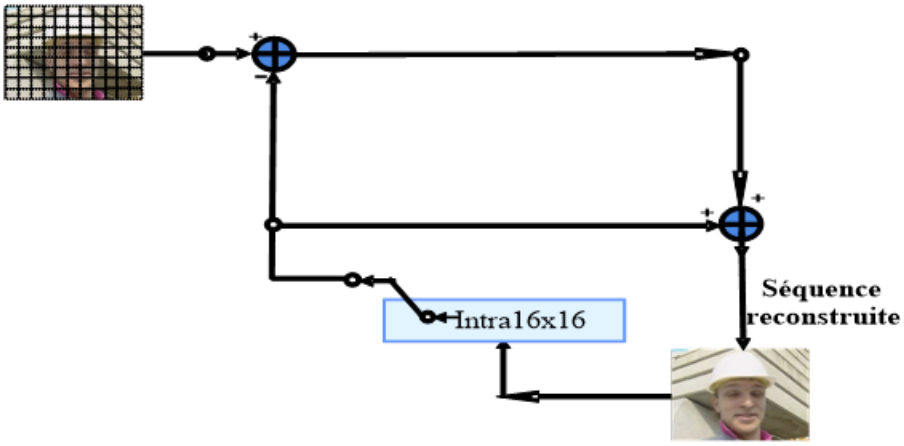


FIGURE 1.3 – Schéma du processus de prédiction intra 16x16

1.4 Le DSP TMS320C6416 : caractéristiques

Le processeur TMS320C6416 [3], appartenant à la famille C64x de Texas Instruments, est un DSP à points fixes particulièrement performant, conçu pour les applications nécessitant des traitements intensifs. Il intègre une mémoire interne de 1 Mo qui permet un accès rapide aux données et aux programmes, ainsi qu'une fréquence d'horloge de 720 MHz offrant une grande capacité de calcul en temps réel. Son architecture, optimisée pour le traitement en point fixe sur 16 bits, assure une exécution rapide et efficace des algorithmes complexes(Figure 1.4). Ces caractéristiques en font un choix privilégié pour l'implémentation d'applications vidéo exigeantes telles que la compression selon la norme H.264/AVC.

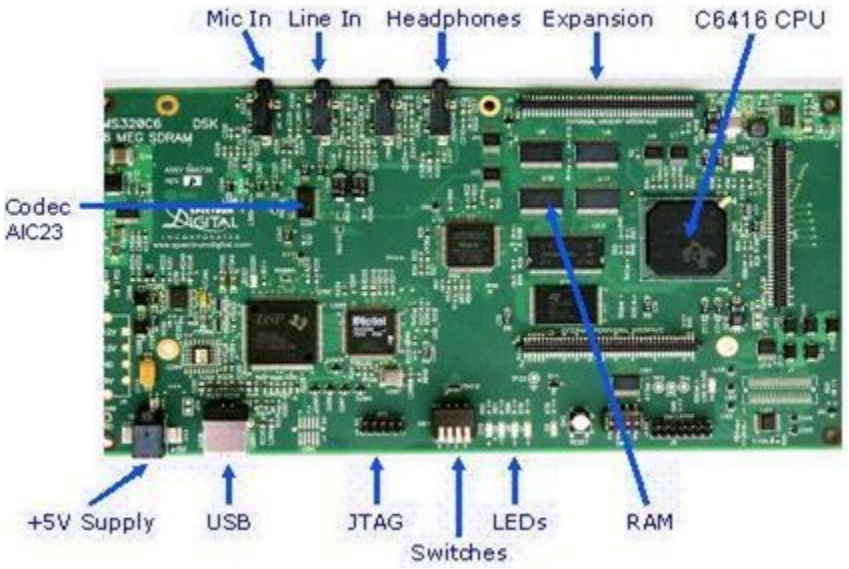


FIGURE 1.4 – Vue d'ensemble de la carte de développement TMS320C6416 DSK illustrant les principaux composants du système DSP [4]

En tirant parti de l'architecture du processeur TMS320C6416 (Figure 1.5), qui combine plusieurs groupes de registres, des unités fonctionnelles parallèles, un jeu d'instructions optimisé, un module de transfert

DMA et une mémoire cache hiérarchisée, les étudiants entreprennent la conception progressive d'un encodeur vidéo H.264 de profil Baseline. La première étape consiste à élaborer le schéma fonctionnel intégrant

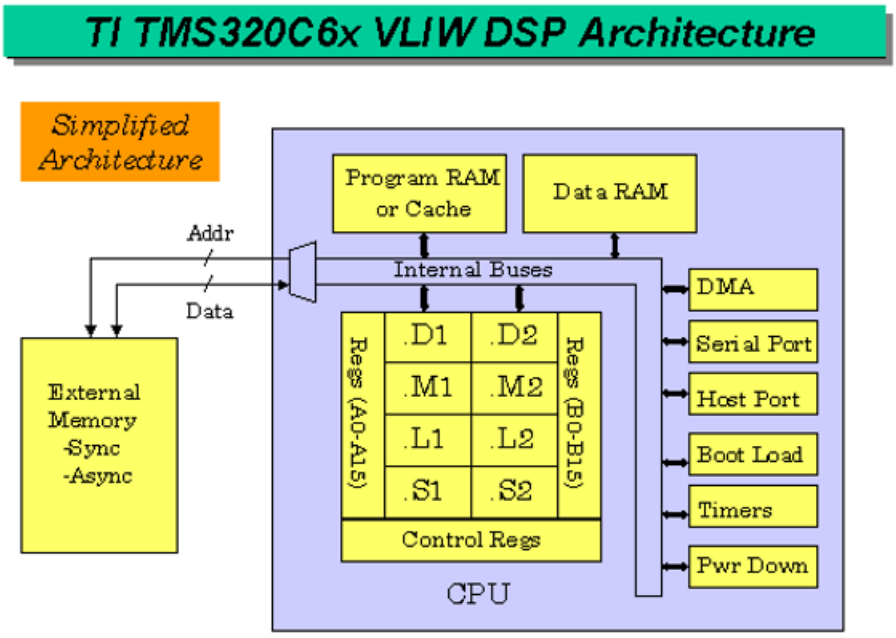


FIGURE 1.5 – Architecture simplifiée du DSP TMS320C6x [5]

le module de prédiction spatiale (Figure 1.6), élément fondamental du codage intra-image. La prédiction INTRA-16x16, particulièrement adaptée aux zones d'image homogènes, repose sur quatre modes principaux — vertical, horizontal, DC et plan —, ce dernier se distinguant par une complexité de calcul plus élevée.

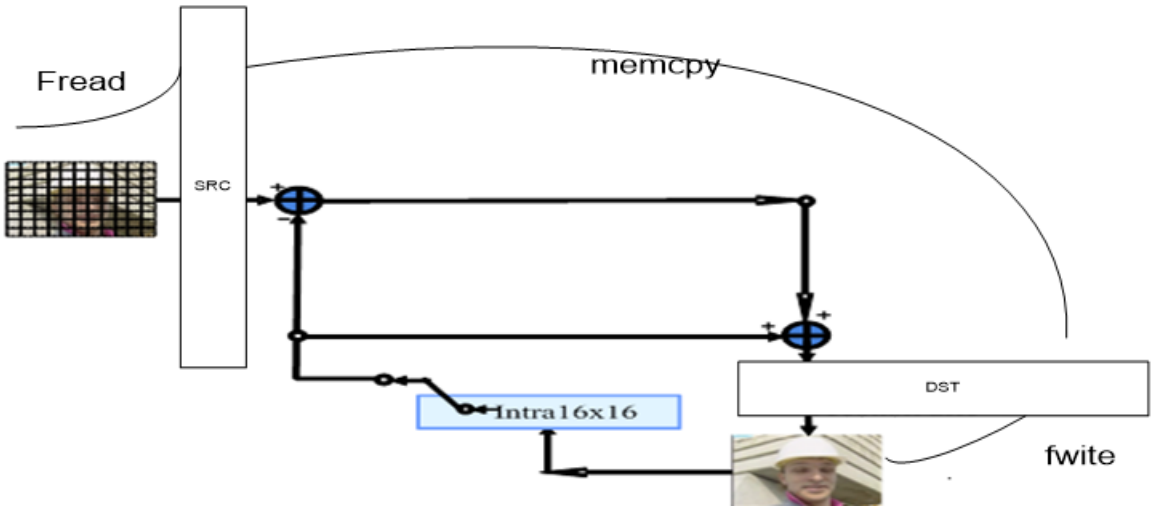


FIGURE 1.6 – Intégration des opérations mémoire dans le flux de traitement vidéo H.264 sur DSP

1.5 Optimisation mémoire : approche Macrobloc par Macrobloc

L'optimisation mémoire du traitement H.264 repose sur une approche « Macrobloc par Macrobloc », où chaque image est découpée en unités indépendantes appelées Macroblocs (MB). Dans cette méthode, le code du standard H.264, d'une taille d'environ 120 Ko, est chargé dans la mémoire interne LL2RAM, laissant environ 488 Ko disponibles pour le stockage temporaire des données. L'encodage est réalisé séquentiellement, bloc après bloc, afin de minimiser l'utilisation mémoire et d'assurer un traitement efficace. Dans le format YUV 4 :2 :0, chaque groupe de quatre pixels de luminance (Y) correspond à un seul pixel de chrominance bleue (U) et un seul pixel de chrominance rouge (V) comme le montre la Figure 1.7, réduisant ainsi la quantité de données à manipuler sans perte visuelle significative.

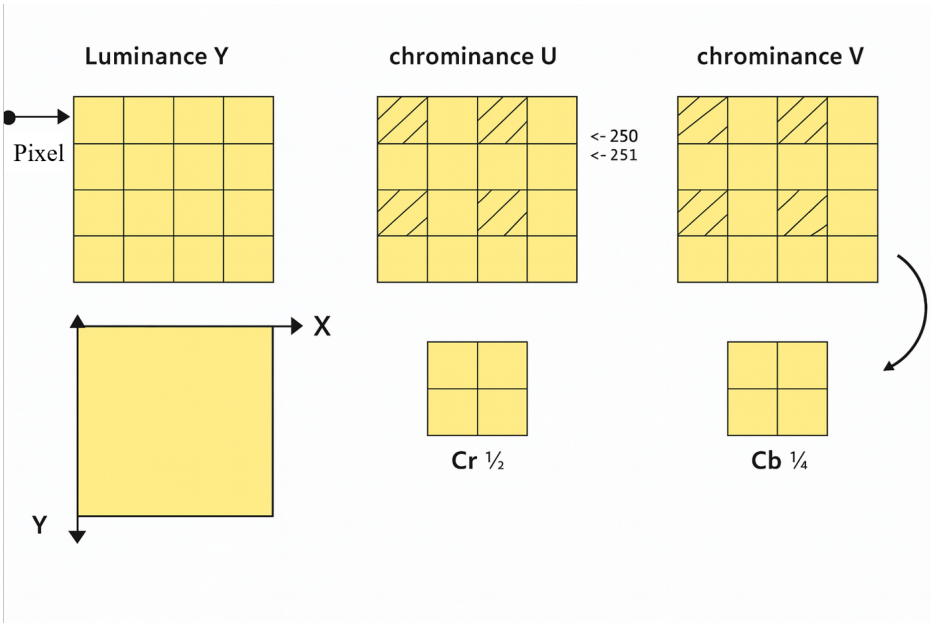


FIGURE 1.7 – Schéma illustrant l'architecture d'une image au format YUV 4 :2 :0

La validation logicielle des algorithmes sous Visual Studio (Figure1.8) suit un processus progressif d'adaptation numérique. Dans un premier temps, les algorithmes sont développés et testés en virgule flottante 64 bits afin de garantir une précision maximale lors des calculs. Ils sont ensuite convertis en représentation à point fixe 32 bits pour s'adapter aux contraintes matérielles et optimiser les performances. Enfin, une dernière adaptation en 16 bits est réalisée afin d'assurer la compatibilité totale avec l'architecture interne du DSP, qui fonctionne principalement sur des opérations en point fixe de cette taille.



FIGURE 1.8 – Visual Studio Code [6]

1.6 Développement

L'objectif de ce travail est d'apprendre à déterminer les adresses mémoire correspondant à des pixels précis dans une image numérique tout en comprenant la répartition des données dans le format YUV 4 :2 :0. Il s'agit également d'observer l'impact du sous-échantillonnage sur la structure mémoire et la gestion des trois plans (Y, U et V). Il s'agit d'assurer le transfert des données stockées dans la mémoire source (SRC) vers la mémoire externe du DSP, conformément au schéma ci-dessous de la Figure 1.9. Le code en C

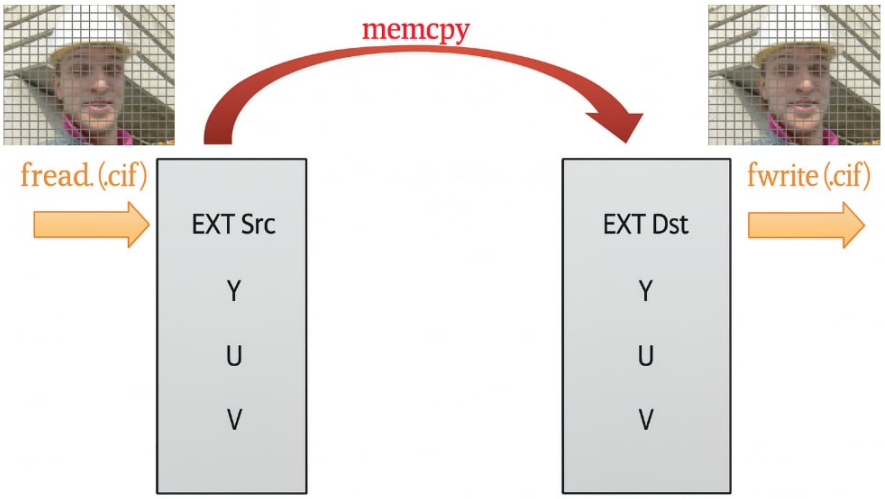


FIGURE 1.9 – Illustration du transfert de données d'une mémoire source (SRC_MEM) vers une mémoire de destination (DST_MEM).

correspondant est le suivant :

```c
/* standard C library includes */
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

#include "LETI_H264_enc.h"

/*#############################################################################################*/
/*                                Encoder parameters                                           */
/*#############################################################################################*/

#define         Input_File          "foreman.cif" // séquence d'entrée
#define         Output_File         "mpp_test.cif" // séquence reconstruite

FILE * fp_input;
FILE * fp_output;

unsigned int    width               = 352; // largeur de l'image cif
unsigned int    height              = 288; // hauteur de l'image cif
unsigned int    Seq_start           = 0;
unsigned int    Seq_length          = 50;  // nombre d'image traitées de la séquence
unsigned char   YUVFormat           = 1;      // YUV format (0=4:0:0, 1=4:2:0, 2=4:2:2, 3=4:4:4)


int main()
{
    unsigned int k,i;
    LETI_H264_EncInfo * pstEncInfo; // pointeur sur la structure LETI_H264_EncInfo

    unsigned int size = (width*height*3)/2;
    // résérvation d'une zone mémoire externe pour loger l'image source src (entrée)
    unsigned char * external_buffer_src = (unsigned char*)malloc(sizeof(unsigned char) * size);
    // résérvation d'une zone mémoire externe pour loger l'image reconstruite dst (sortie)
    unsigned char * external_buffer_dst = (unsigned char*)malloc(sizeof(unsigned char) * size);
    pstEncInfo = (LETI_H264_EncInfo*)malloc(sizeof(LETI_H264_EncInfo));
    memset(  external_buffer_dst,255,size);

    fp_input  = fopen(Input_File,  "rb");// ouvrir le fichier d'entrée en mode lecture
    fp_output = fopen(Output_File, "wb");// ouvrir le fichier d'entrée en mode ecriture

    pstEncInfo->width = width;//352
    pstEncInfo->Chroma_width    = width>>1;// 352/2= 176
    pstEncInfo->height = height;//288
    pstEncInfo->FrameNum = 0;
    pstEncInfo->Max_MBX         = width>>4;//   352/16= 22
    pstEncInfo->Max_MBY         = height>>4;//  288/16= 18

      printf("H264BP_Desktop\n");

    for (k = 0; k < Seq_length; k++)
        //         DST;              pas; Taille; Src
    {
        if (fread(external_buffer_src, 1, size, fp_input) < size)
            break;

        { // pinteur src
            // Read one frame 4:2:0 Cif size to src
            pstEncInfo->pSrcFrameY = external_buffer_src;
            pstEncInfo->pSrcFrameU = external_buffer_src + width*height;
            pstEncInfo->pSrcFrameV = pstEncInfo->pSrcFrameU + width*height/4;
// pointeur  dst
```

```
62
63              pstEncInfo->pDstFrameY= external_buffer_dst;
64              pstEncInfo->pDstFrameU = external_buffer_dst + width*height;
65              pstEncInfo->pDstFrameV = pstEncInfo->pDstFrameU + width*height/4;
66              printf("Processing Frame %d\t",k);
67              //// memcpy(dst, src, quantit);
68              pstEncInfo->pDstFrameY;
69
70               for(i = 0; i<pstEncInfo->height; i++)//288
71      {//             DST                          SRC                                    Taille
72          memcpy(pstEncInfo->pDstFrameY+(pstEncInfo->width * i),
73          pstEncInfo->pSrcFrameY+(pstEncInfo->width * i), pstEncInfo->width);
74      }
75
76      for(i = 0; i<pstEncInfo->height/2; i++) //144
77      {
78          memcpy(pstEncInfo->pDstFrameU+(pstEncInfo->Chroma_width * i),
79          pstEncInfo->pSrcFrameU+(pstEncInfo->Chroma_width * i) , pstEncInfo->Chroma_width);
80      }
81
82      for(i = 0; i<pstEncInfo->height/2; i++)
83      {
84          memcpy(pstEncInfo->pDstFrameV, pstEncInfo->pSrcFrameV +(pstEncInfo->Chroma_width * i),
85          pstEncInfo->Chroma_width);
86          pstEncInfo->pDstFrameV += pstEncInfo->Chroma_width;
87      }
88
89              }
90          pstEncInfo->FrameNum++;
91
92          //     SRC           pas     taille          Dst
93          fwrite(external_buffer_dst, 1, width*height*3/2, fp_output);
94      }
95      // libérer les allocations mémoires
96      free(external_buffer_src);
97      free(external_buffer_dst);
98      free(pstEncInfo);
99
100     exit(0);
101
102 }
103
```

1.6.1 Rappel : calcul d'adresse en mémoire

Dans le format YUV 4 :2 :0, les trois plans (Y, U, V) sont organisés différemment en mémoire. Le plan Y (luminance) contient la totalité des pixels de l'image, tandis que les plans U et V (chrominance) sont sous-échantillonnés d'un facteur deux dans les directions horizontale et verticale, ce qui signifie qu'ils contiennent chacun quatre fois moins d'informations que le plan Y. Les adresses mémoire de chaque pixel sont déterminées à partir des coordonnées du point à traiter (X,Y)(X, Y)(X,Y) et de la largeur de l'image (ici 352 pixels, selon le format CIF standard). Pour le plan de luminance Y :

$$@P_Y = @P0 + (Ylargeur) + X \tag{1.1}$$

Pour les plans de chrominance U et V :

$$@P_{U/V} = @P0 + (Ylargeur) + (X) \tag{1.2}$$

Ces formules permettent de localiser précisément l'adresse mémoire correspondant à chaque pixel dans les

différents plans de l'image.

1.6.2 Travail demandé

Le travail pratique consiste à appliquer ces formules afin de calculer les adresses mémoire correspondant à des points donnés dans deux trames distinctes.

— Dans la frame 0, on détermine les adresses associées au point (45, 120) pour les trois plans : Y, U et V.

— Dans la frame 7, la même opération est effectuée pour le point (200, 60). Ces calculs permettront d'identifier la position exacte de chaque composante de couleur dans la mémoire et d'analyser la manière dont les données vidéo sont organisées dans un flux YUV.

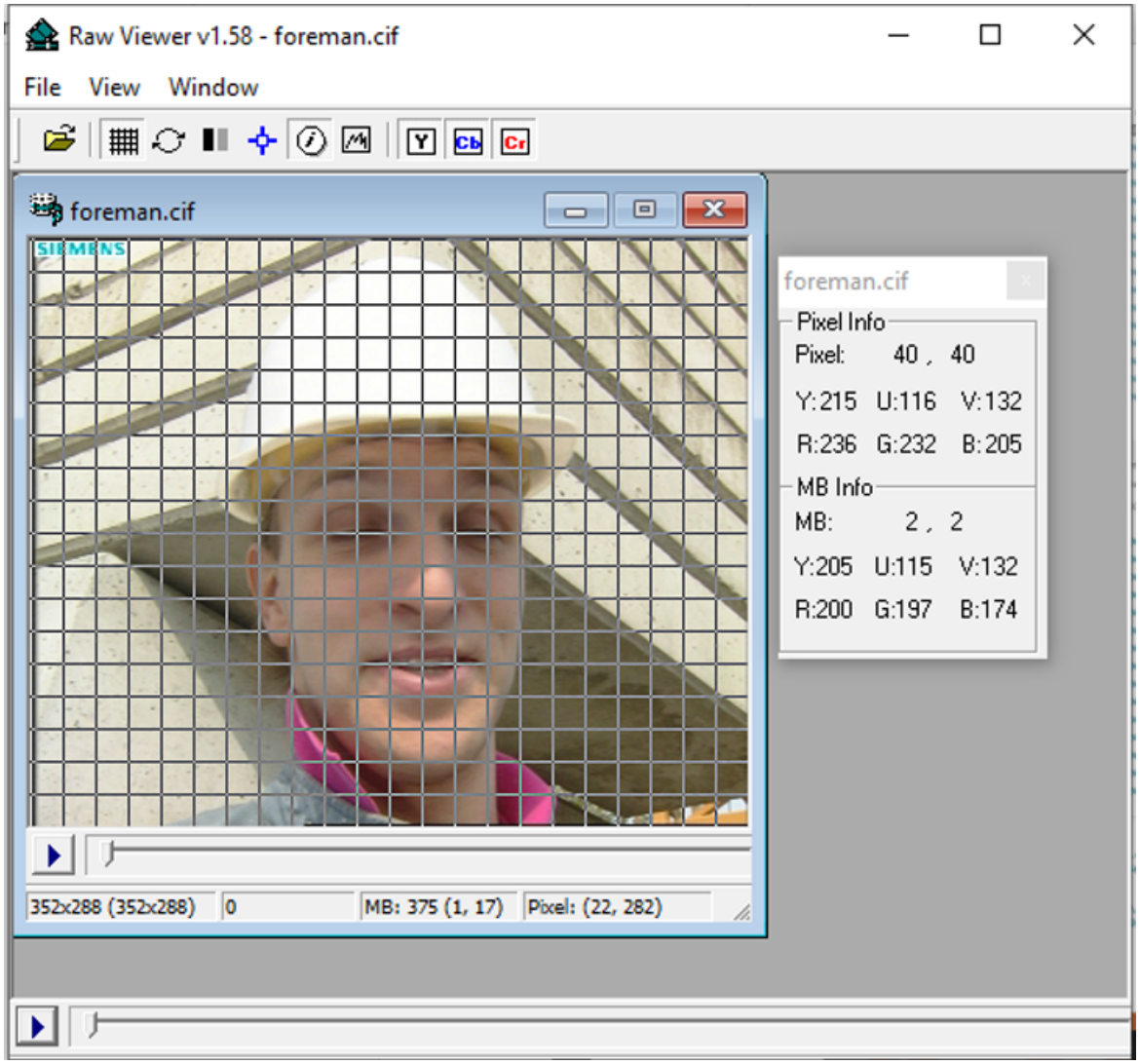
TABLEAU 1.2 – Les coordonnées à la Source

Plan	Adresse	Valeur
SRC_Y	0x00ec470	0A
SRC_ U	0x00105070	80
SRC_V	0x0010b370	81

TABLEAU 1.3 – Les coordonnées à la Destination

Plan	Adresse	Valeur
DST_Y	0x008352b0	0A
DST_U	0x0084deb0	80
DST_V	0x008541b0	81

Exemple complet : calcul des adresses pour la Frame 0, point (40,40)



Considérons un point arbitraire choisi uniquement à titre d'illustration :
(X, Y) = (40,40)
La résolution reste 352×288, format YUV 4 :2 :0.

1. Adresses de base de la Frame 0 Pour la frame 0, l'adresse de départ est :

   • Adresse base frame 0 : 0

   Taille des différents plans au format YUV 4 :2 :0 :

   — Plan Y : 352×288 = 101 376 octets

   — Plan U : 101 376 / 4 = 25 344 octets

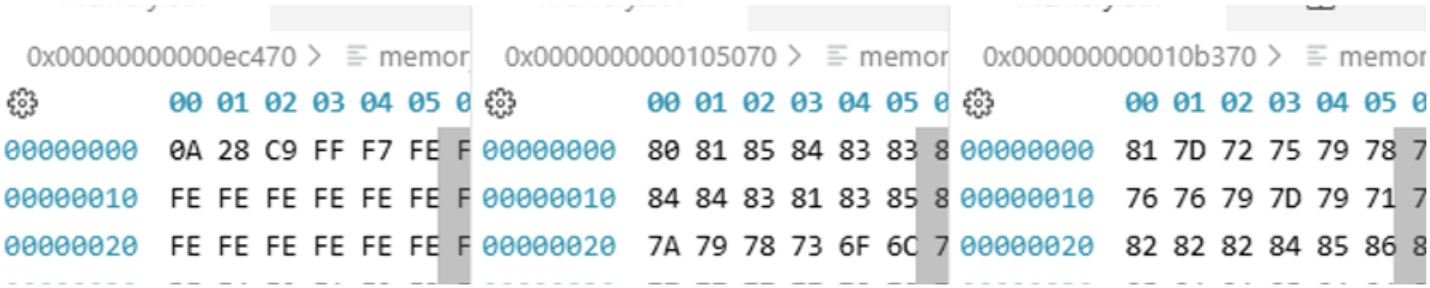
   — Plan V : 25 344 octets (même taille que U)

   Adresses de base :

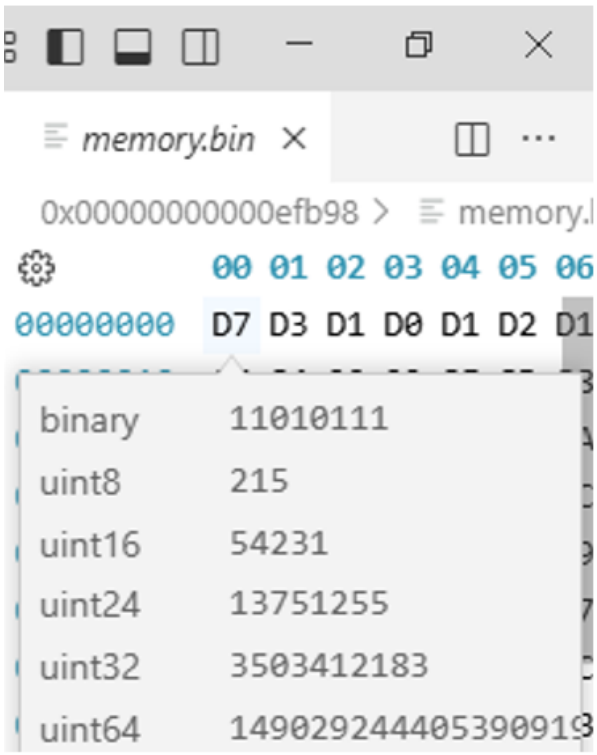
— @P_Y = 0x00ec470

— @P_U = 0x00105070

— @P_V = 0x0010b370



2. Calcul pour le plan Y (luminance) Formule : @P_Y = @P0 + (Y×Largeur) + X
   Application pour X = 40 et Y = 40 :
   @P_Y = 0x00ec470 + (40×352) + 40
   40×352 = 14 080 => 3700 en hexa
   @P_Y = 0x00ec470 + 14 080 + 40 = 0x00ec470 + 3728 = **EFB98 ce qui correspond à la valeur 215**



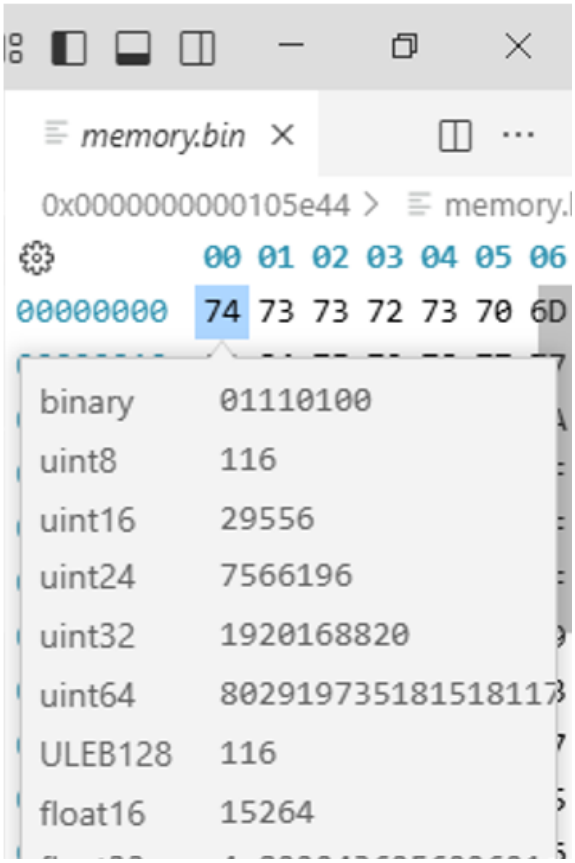
3. Calcul pour le plan U (chrominance bleue) Sous-échantillonnage 4 :2 :0 : X_U = 40 / 2 = 20 Y_U = 40 / 2 = 20 Largeur du plan U = 352 / 2 = 176 Formule : @P_U = @P0 + (Y_U × 176) + X_U

Application :

$@P_U$ = 0x00105070 + (20×176) + 20 20×176 = 3 520 => DC0 en hexa

$@P_U$ = 0x00105070+ 3 520 + 20 = 0x00105070 + DD4 = 105 E44

$@P_U$ (40,40) : 105 E44 ce qui correspond à la valeur 116

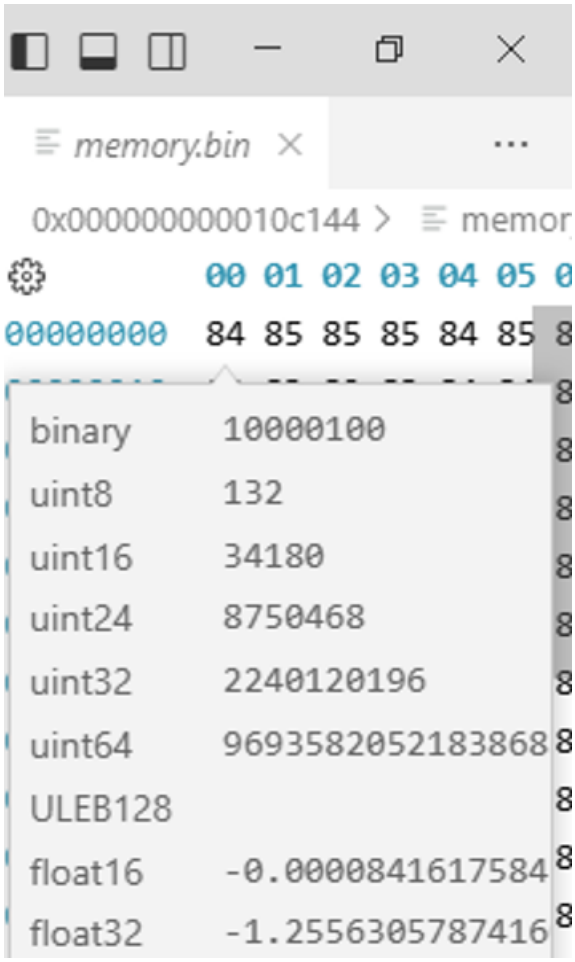


4. Calcul pour le plan V (chrominance rouge) Formule identique au plan U : $@P_U$ = @P0 + (Y_U × 176) + X_U

   Application :

   $@P_V$ = 0x0010b370 + 3 520 + 20 = 0x0010b370 + DD4 = 10C144

   $@P_V$ (20,20) : 10C144 => ce qui correspond à la valeur 132


memory.bin
0x000000000010c144 > memor
00 01 02 03 04 05 0
00000000 84 85 85 85 84 85
binary 10000100
uint8 132
uint16 34180
uint24 8750468
uint32 2240120196
uint64 9693582052183868
ULEB128
float16 -0.0000841617584
float32 -1.2556305787416